VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



SOFTWARE ARCHITECTURE (CO3017)

ASSIGNMENT 2

QR Attendance & Access Control

Architecture Refinement and Edge Implementation

Advisor(s):	Truong Thi Thai Minh	
Student(s):	Nguyen Ngoc Kim	2352661
	Nguyen Nhat Hieu	2352230
	Bui Doan Minh Nhat	2352855
	Nguyen Tran Tuan Khai	2452514
	Vu Quoc Viet	2353326

Ho Chi Minh City, November 2026

Contents

1 System Context, Scope, and Requirements **1**
1.1 Project Context and A2 Objectives 1
1.2 Stakeholders and System Boundary 1
1.3 Functional Requirements 2
1.4 Non-Functional Requirements and Architectural Characteristics 2

2 M2 Issues and Architecture Refinement **3**
2.1 M1 Architecture Baseline 3
2.2 Issue 1: Connectivity Loss During Attendance Processing 3
2.2.1 Failure scenario and architectural impact 3
2.2.2 Refined solution: offline-first local processing 4
2.2.3 Trade-offs and limitations 4
2.3 Issue 2: Exposure and Reuse of Static QR Codes 5
2.3.1 Threat scenario 5
2.3.2 Refined solution: signed and locally verified QR 5
2.4 Architecture Refinement Summary 7

3 Architecture Styles, Views, and Interfaces **7**
3.1 Architecture Style Reassessment 7
3.2 Module View: Static Decomposition 8
3.3 Component-and-Connector View: Runtime Interactions 8
3.4 Allocation View: Software-to-Environment Mapping 9
3.5 Shared Interface Contracts 9

4 Detailed Edge Module Design and Data Model **10**
4.1 Main Module Selection 10
4.2 Access Attempt and Attendance Lifecycle 10
4.3 Class Design and Dependency Inversion 11
4.4 Local Storage and Physical ERD 12
4.5 Persistence and Event Semantics 12

5 Implementation of the Selected Main Module **12**
5.1 Technology and Repository Organization 12
5.2 QR Verification Component 13
5.3 Access Coordination and Local Policy 13
5.4 Turnstile, Persistence, and Offline Synchronization 13
5.5 Source Code Repository and Demonstration Availability 13

6 Verification and Evaluation **14**
6.1 Testing Strategy 14
6.2 Repository Verification Status 14
6.3 Latest Reported Automated Test Results 15
6.4 Three Representative Interactive Demo Scenarios 15
6.5 Requirements Traceability 17

7 Architecture Decisions and SOLID Principles **17**
7.1 ADRs Carried Forward from Assignment 1 17
7.2 Architectural Ambiguities and Working Assumptions 17
7.3 SOLID in the Implemented Edge Module 18
7.4 SOLID Reflection: Benefits and Challenges 18

8 Conclusion and Known Limitations **19**

References **20**

A AI Tool Usage Declaration **21**
A.1 Nguyen Tran Tuan Khai — Architecture Views and Interface Contracts . . 21
A.2 Nguyen Ngoc Kim — Edge Access Core and Policy Evaluation 21
A.3 Nguyen Nhat Hieu — Issue Analysis and QR Verification 21
A.4 Bui Doan Minh Nhat — Persistence, Turnstile and Synchronization 21
A.5 Vu Quoc Viet — ADR Refinement, Integration and Report Assembly . . . 22

1 System Context, Scope, and Requirements

1.1 Project Context and A2 Objectives

TMC Manufacturing Joint Stock Company operates two factories in Binh Duong (approximately 800 employees) and Long An (approximately 600 employees). The proposed QR-based Attendance and Access Control System verifies employee credentials at entry and exit gates, evaluates access permissions, communicates with existing turnstile controllers, and records transactions for attendance management. The system is designed to integrate with an existing HR/Payroll platform, with attendance data available for reconciliation before 12:00 PM on the first day of each month [1, 2].

Assignment 1 (M1) established the architectural baseline. Assignment 2 (M2) refines this design in response to two issues: connectivity loss during attendance processing and exposure/reuse of static QR codes. M2 also requires documenting three architecture views, applying and reflecting on SOLID principles, and implementing at least one main module [1]. The group's implementation focus is the **Edge Gate Access Processing** module. This is an intentionally bounded prototype, not an assertion that the entire TMC enterprise platform has been built.

1.2 Stakeholders and System Boundary

The identified actors are employees, production supervisors, HR staff, security guards, system administrators, and executive management. Employees display QR credentials through a mobile device. The access-control system interacts with the existing turnstile controller and the existing HR/Payroll system without replacing them. Visitor management and payroll calculation are outside scope [1].

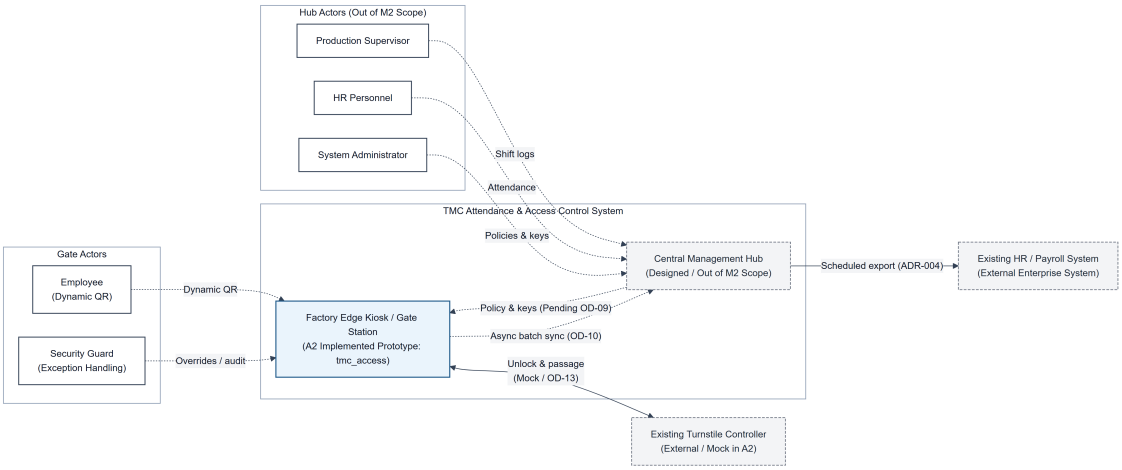


Figure 1: System context and boundary of the TMC Attendance and Access Control System.

1.3 Functional Requirements

Requirement group	**System responsibility**	**A2 prototype boundary**
User and QR management	Issue, validate and revoke employee credentials	Edge verification implemented; issuance/rotation at Hub not implemented in the prototype
Gate station management	Associate scanning, permissions and gate commands with stations	Simulated scan input and gate adapter; no physical hardware
Attendance event recording	Audit access attempts and create attendance events under confirmed criteria	Edge coordinator and SQLite-backed attendance/outbox implementation
Reporting and auditing	Retain evidence and support later reconciliation/reporting	Local audit and synchronization simulation; central reporting outside A2 prototype
System administration	Policies, identities, configuration and observability	Local policy snapshots/configuration represented; full administration system outside implementation

Table 1: Functional requirement groups and implementation boundaries (M2 brief and current prototype).

1.4 Non-Functional Requirements and Architectural Characteristics

The M2 brief defines four principal constraints: kiosk response time no greater than two seconds under the stated workload, no attendance-record loss in normal operation, RBAC with comprehensive audit logging, and protection of employees' personal data [1]. At shift change, a factory may process approximately 800 transactions in 30 minutes [1]. Important architectural characteristics are therefore performance, reliability, security, maintainability, and interoperability.

Quality attribute	**Design response**	**Verification / status**
Performance	Local QR verification and policy evaluation; bounded turnstile operation	End-to-end ≤ 2 s remains unverified without a representative timed workload; a 15 s passage window is a separate workflow bound
Reliability	File-backed SQLite WAL, durable attempts, transactional attendance/outbox, asynchronous retry	Local and simulated-sync tests reported; not a production power-loss guarantee
Security	Signed dynamic QR, issuer/audience/expiry checks, replay guard, local deny rules	QR unit tests reported; cross-station replay and key issuance remain outside prototype
Maintainability	Layered modules, domain ports, dependency injection and deterministic tests	Class/interface analysis and test evidence
Interoperability	GateAdapter and SyncClient interfaces separate application logic from vendors/Hub	Mock turnstile and simulated Hub; real vendor protocols pending

Table 2: Architectural characteristics and evidence boundaries.

2 M2 Issues and Architecture Refinement

2.1 M1 Architecture Baseline

The A1 design selected a distributed **Edge–Hub topology**, a **Modular Monolith** for the Central Hub, and layered organization inside the Edge and Hub [2]. The Edge is designed to make selected gate decisions without a synchronous Hub dependency. Its main elements include QR validation, access control, gate adaptation, local storage, a transactional outbox, and a synchronization worker. The Central Hub remains the enterprise-side system for management, issuance, ingestion, reporting, and HR/Payroll integration. A2 implements the selected Edge module, not a separate Central Hub service.

2.2 Issue 1: Connectivity Loss During Attendance Processing

2.2.1 Failure scenario and architectural impact

WAN connectivity can fail before, during, or after a gate transaction. Depending on the failure point, a synchronous Hub-only design can delay the gate, lose an unsent attendance event, or retransmit an event whose acknowledgement was lost. The design must distinguish the local access decision from eventual delivery to the central system.

2.2.2 Refined solution: offline-first local processing

The Edge verifies QR credentials with cached public keys and evaluates a local policy snapshot. The implementation of the policy evaluator follows A1's age windows: a snapshot no older than five minutes is fresh; a snapshot older than five minutes but no older than two hours is usable with a stale-cache indicator; an older or missing snapshot is rejected. This policy is not equivalent to a complete Hub policy-distribution implementation.

For successful physical passage, the Edge writes the attendance record and its outbox entry in one local SQLite transaction. A separate sync service sends pending events when a Hub-like receiver is available. The simulated receiver uses stable transaction identifiers to handle repeated delivery and lost acknowledgements, while the retry worker applies configured backoff. The kiosk access path is not required to wait for successful central synchronization.

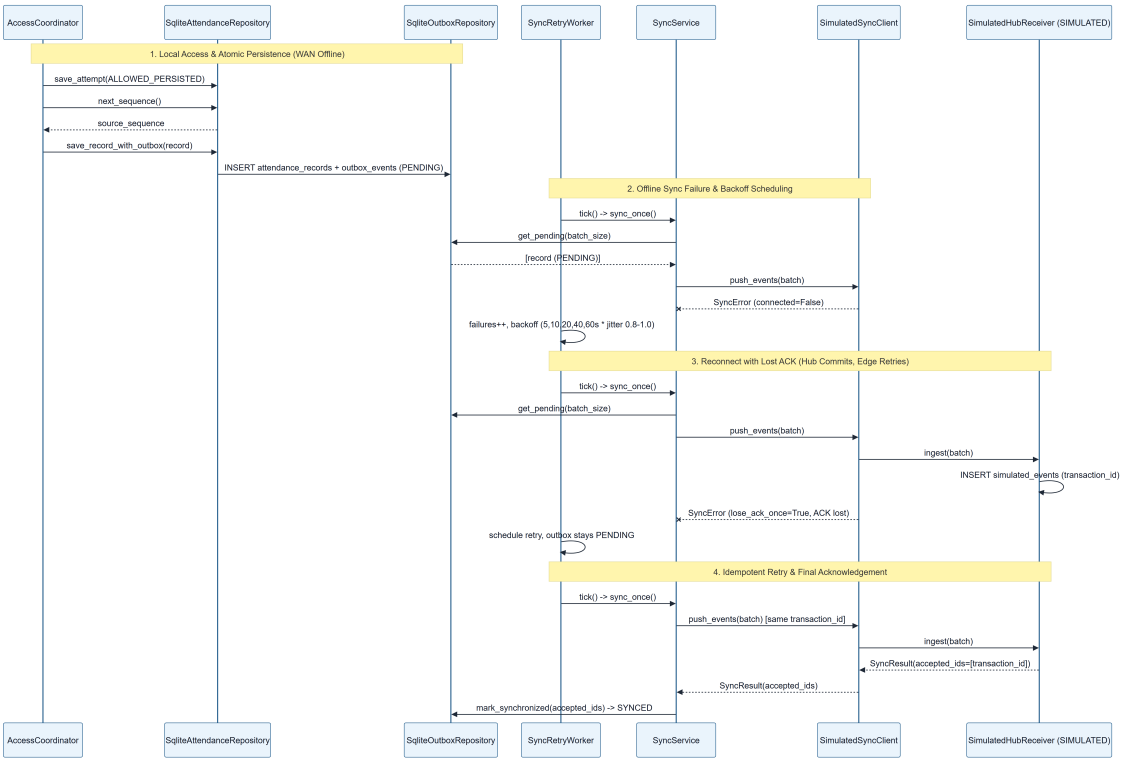


Figure 2: Offline persistence and reconnect synchronization sequence (prototype).

2.2.3 Trade-offs and limitations

Local operation reduces WAN dependency at the cost of managing stale authorization data, durable storage, clock quality and duplicate transmission. The A2 prototype uses a simulated Hub rather than a production ingestion service. No claim of production-grade outage availability or zero loss under all hardware failures is made without corresponding measurements.

2.3 Issue 2: Exposure and Reuse of Static QR Codes

2.3.1 Threat scenario

A static QR credential can be photographed or shared and subsequently reused. A digital signature by itself protects integrity and provenance, but a captured *valid* token may still be presented within its validity window. The mitigation must combine short lifetime, trusted issuer/key checks, audience restrictions and anti-replay state.

2.3.2 Refined solution: signed and locally verified QR

The implemented Edge verifier uses Ed25519 public-key verification and validates QR claims including issue/expiry times, station audience, issuer and key identifier. The present prototype uses a custom signed token represented as two encoded parts, not standard three-part JWS Compact Serialization. Public verification keys are held on the Edge; the private signing/issuance flow is not implemented in this module.

The `ReplayGuard` abstraction records consumed token identifiers and rejects repeated validation in the current process. This establishes a testable local security control but does not establish persistence across restart or coordinated anti-replay among kiosks. The merged verifier checks issuance and expiry timestamps but does not impose a separate maximum value for `exp - iat`; short-lifetime enforcement and final token issuance/format remain architectural follow-up decisions.

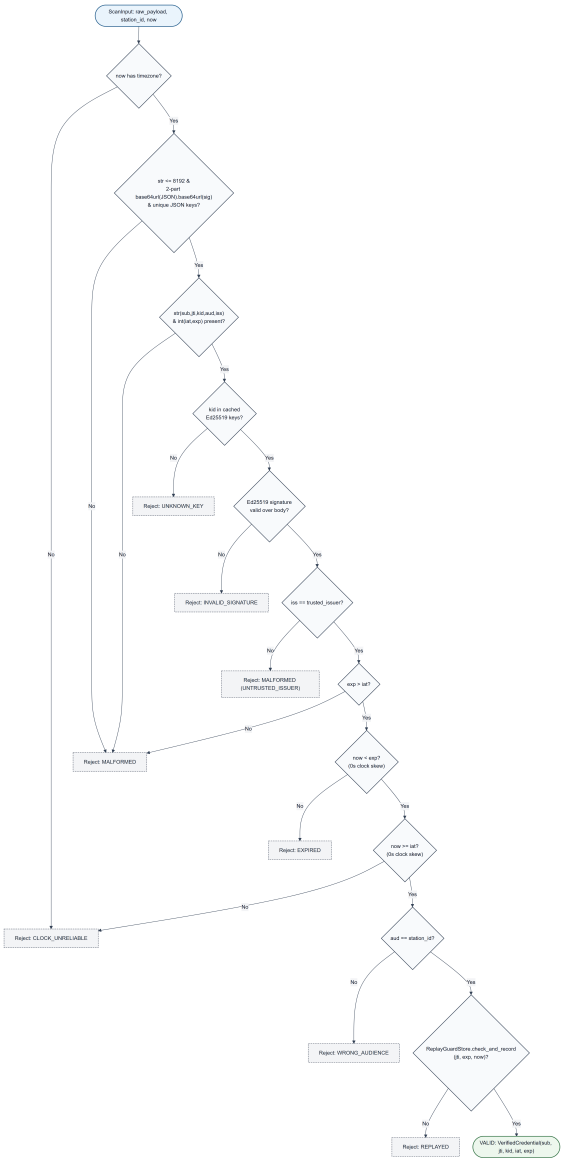


Figure 3: QR validation and replay decision flow at the Edge.

2.4 Architecture Refinement Summary

Concern	A1 design intent	A2 evidence / refinement
QR security	Dynamic signed QR, public-key checking at Edge	Ed25519 verifier and local replay guard; issuance remains out of scope
Access sequencing	Access attempt, unlock, passage and attendance events	Implementation separates durable attempt, gate ACK, passage and attendance; A1 ambiguity explicitly recorded
Network resilience	SQLite WAL, transactional outbox and store-and-forward	SQLite repositories, retry worker and simulated Hub synchronization
Modularity	Layered Edge with defined interfaces	Application uses domain ports; infrastructure adapters implement ports

Table 3: Continuity from A1 design to the bounded A2 implementation.

3 Architecture Styles, Views, and Interfaces

3.1 Architecture Style Reassessment

The two M2 issues strengthen the rationale for the existing Edge–Hub design rather than requiring a new macro architecture. The trade-off analysis considers coupling, operational resilience and integration cost [5]. A Hub-only transaction path would directly depend on WAN availability; an Edge-only isolated system would complicate central management and reconciliation. The Edge–Hub separation allows local gate decisions while preserving an eventual path for enterprise synchronization. At the Hub, the accepted Modular Monolith remains a design decision from A1, not an implemented claim in A2. Within the Edge, domain abstractions and layered dependencies support independent adaptation of QR verification, turnstile access, and persistence.

Alternative	Trade-off relevant to M2	Assessment
Central-only processing	Simpler central ownership, but gate path depends on WAN	Does not adequately address outage continuity
Fully independent kiosks	Strong local autonomy, but difficult central coordination	Does not by itself meet reporting/integration needs
Distributed Edge–Hub	Local decisions and eventual reporting, at cost of synchronization complexity	Retained architectural choice from A1

Table 4: Reassessment of architecture choices against the two M2 issues.

3.2 Module View: Static Decomposition

The module view represents implementation units and static dependencies, following the views-and-beyond approach to architectural documentation [3]. The Edge code is organized into `domain` (models, ports and policies), `application` (access orchestration and synchronization use cases), `infrastructure` (QR verification, turnstile adapter, SQLite persistence and sync simulation), `presentation` (API), and `config` (composition root). The domain does not import infrastructure/framework code; application logic depends on abstractions; concrete adapters are wired by the composition root.

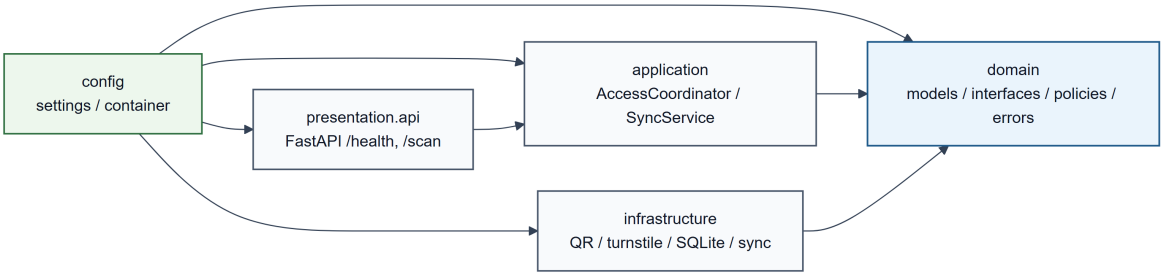


Figure 4: Module view of the implemented Edge Gate Access Processing module.

3.3 Component-and-Connector View: Runtime Interactions

At runtime, a scan request reaches the Edge API, which invokes the AccessCoordinator. The coordinator calls QR verification and policy evaluation, records an access attempt, interacts with the GateAdapter, evaluates passage evidence, and persists an attendance record with the outbox when the success condition is satisfied. Synchronization runs separately from this synchronous gate path. A kiosk display or physical feedback actuator is conceptual unless implemented in the integrated source.

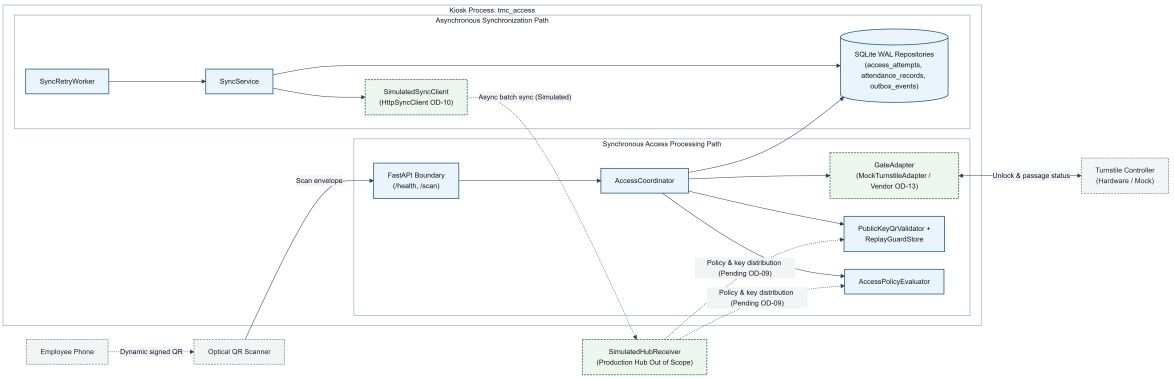


Figure 5: Component-and-connector view of runtime Edge access and synchronization.

3.4 Allocation View: Software-to-Environment Mapping

The A1 architecture defines User Device, Factory Gate Station and Central Server zones. In A2, Python Edge components, the API simulation boundary and file-backed SQLite are implemented for the factory kiosk context. A simulated turnstile and simulated Hub substitute for physical vendor devices and backend systems. The actual production Hub and HR/Payroll integration are documented as architectural context, not delivered runtime components.

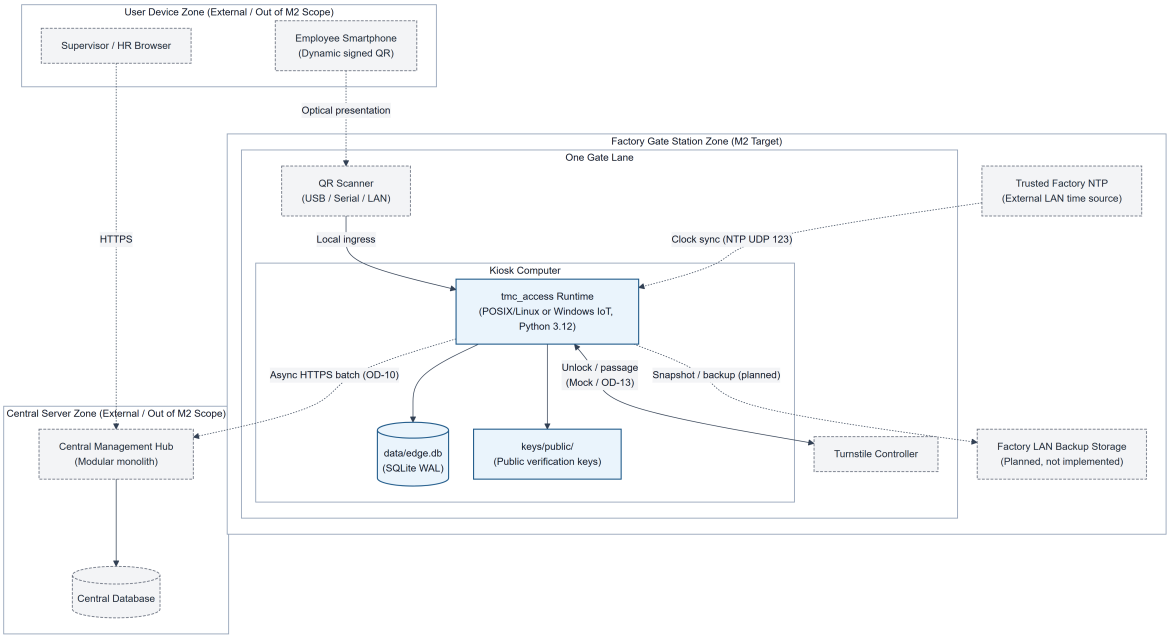


Figure 6: Allocation view of the TMC system and the A2 prototype boundary.

3.5 Shared Interface Contracts

The application defines separate contracts for QR validation, policy evaluation, gate commands and passage queries, attendance persistence, outbox storage and synchronization. These are exposed through `QrValidator`, `AccessPolicyEngine`, `GateAdapter`, `AttendanceRepository`, `OutboxRepository` and `SyncClient`. The merged code standardizes `AttendanceRecord` metadata including `controller_ack`, `policy_version` and `source_sequence`. The SQLite adapter provides `next_sequence()`, currently computed from the largest stored attendance sequence. Its behavior under concurrent in-flight scans remains a known limitation, rather than a proven unique-sequence allocator. The contracts document types, failure behavior and persistence semantics at each component boundary.

Port/interface	Responsible module	Boundary / evidence
`QrValidator`	QR infrastructure	Validity verdict plus verified credential
`AccessPolicyEngine`	Domain policy	ALLOW/DENY and policy metadata
`GateAdapter`	Turnstile infrastructure	Command acknowledgement separate from passage
`AttendanceRepository`	SQLite infrastructure	Durable attempt; atomic record and outbox
`OutboxRepository`	SQLite infrastructure	Pending records and sync-state transitions
`SyncClient`	Synchronization infrastructure	Batch transfer, ACK/duplicate/conflict result

Table 5: Major Edge ports and their implementation boundaries.

4 Detailed Edge Module Design and Data Model

4.1 Main Module Selection

The group selected Edge Gate Access Processing as the main implemented module for M2 Task 2. The module contains the decision path and its required supporting adapters. It does not include a complete mobile credential issuer, real Hub, actual turnstile SDK or enterprise payroll exporter. Supporting simulation components exist so that failure and integration scenarios can be tested without production equipment.

4.2 Access Attempt and Attendance Lifecycle

A QR scan does not directly imply attendance. The coordinator first obtains a QR verdict and a policy decision. If allowed, it persists an access attempt before issuing the gate unlock command. Controller ACK is not physical passage. Attendance is created only when the passage-confirmation condition adopted for the A2 prototype is met. The outbox write accompanies the attendance record in one local transaction; central ACK is a later stage. This sequence follows the A2 prototype's documented working assumption for the attendance lifecycle.

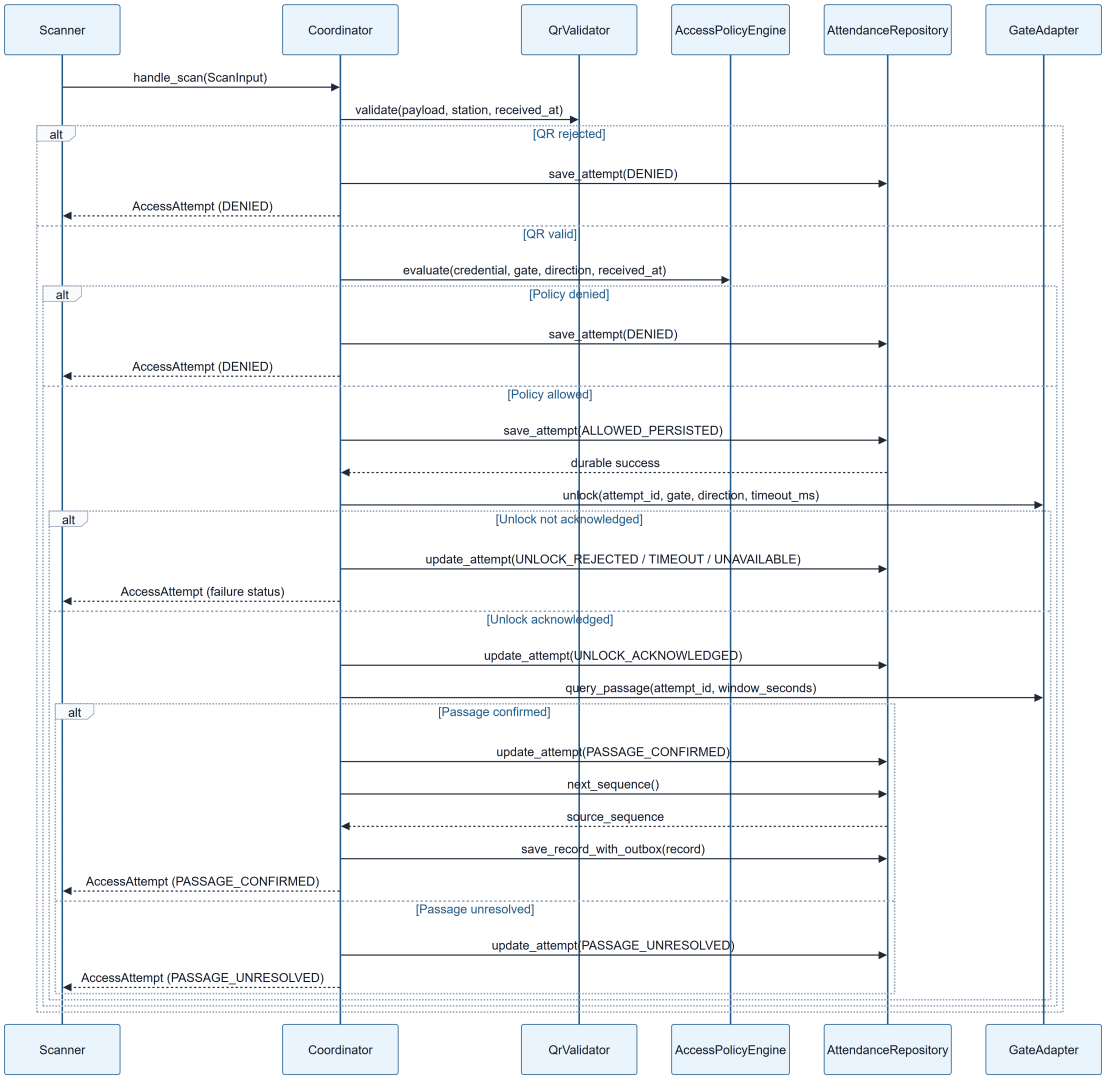


Figure 7: Access processing sequence and failure branches (A2 prototype).

4.3 Class Design and Dependency Inversion

`AccessCoordinator` orchestrates the use case; it does not implement cryptographic verification or SQL itself. `AccessPolicyEvaluator` evaluates rights against cached policy snapshots. `PublicKeyQrValidator` implements local signature/claim checking, while `ReplayGuardStore` owns the consumed-token state. The gate, repository and sync implementations remain behind domain protocols. Operational timeouts and windows are supplied through settings/composition root.

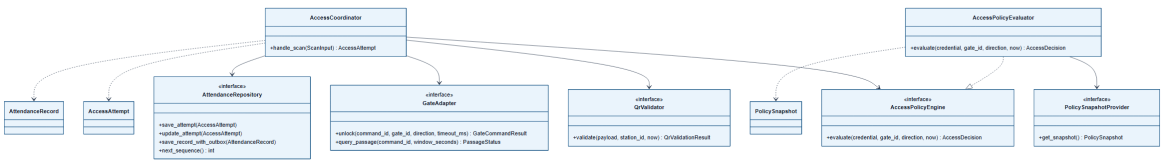


Figure 8: Class and interface relationships of the Edge Access Core.

4.4 Local Storage and Physical ERD

The prototype's persistence schema includes access attempts, attendance records and transactional outbox events. The attendance record refers to an access attempt; the outbox entry uses the attendance transaction identifier. SQLite WAL and foreign-key enforcement are implementation-level choices. The physical schema is defined by the integrated `CREATE TABLE` statements rather than inferred from the abstract domain model. The merged schema has no separate sequence-counter table; therefore the ERD must not invent one.

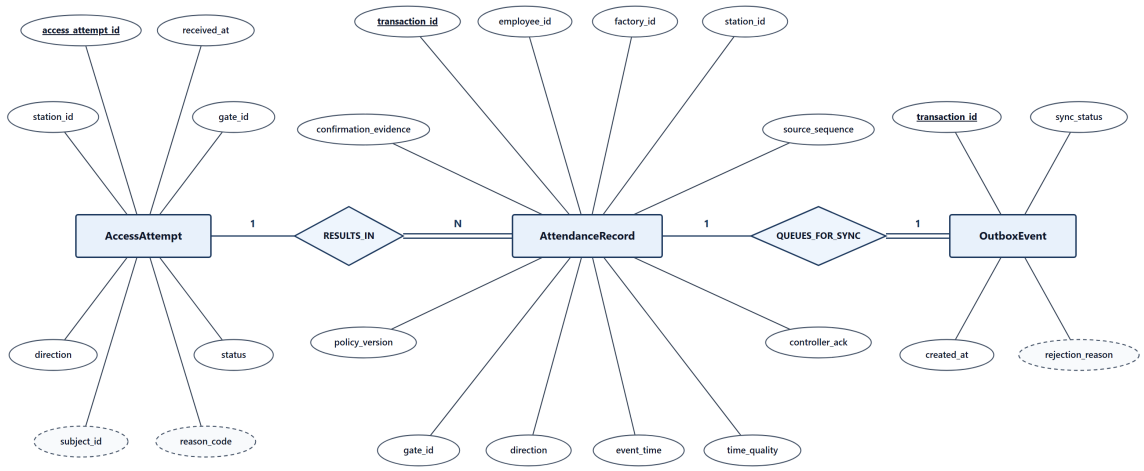


Figure 9: Physical ERD of the final Edge SQLite schema.

4.5 Persistence and Event Semantics

A successful passage yields a uniquely identified attendance transaction. Local persistence is intended to commit its attendance record and pending outbox entry together, preventing the case in which attendance exists without a corresponding item ready for central delivery. Synchronization is modeled as at-least-once; stable transaction IDs and receiver-side idempotency mitigate duplicated storage when network ACKs are lost. The prototype receiver is simulated; production Hub ingestion behavior is not claimed as implemented.

5 Implementation of the Selected Main Module

5.1 Technology and Repository Organization

The integrated implementation at repository commit `a2a9195` (14 November 2026) uses Python 3.12 with a `src/tmc_access` layout. The domain layer defines models, policies, and ports; application services orchestrate access processing and synchronization; infrastructure supplies Ed25519 QR verification, a mock gate, SQLite repositories, and synchronization adapters; presentation exposes a FastAPI scan-simulation boundary. Dependency injection

is handled by the configuration/composition root. The project declares FastAPI, Uvicorn, Pydantic, pydantic-settings, and cryptography dependencies in `pyproject.toml`, with pytest and Ruff for development validation.

The A2 implementation covers the Edge prototype; mobile credential issuance, the production Hub and physical turnstile integration remain outside its scope.

5.2 QR Verification Component

The implemented `PublicKeyQrValidator` verifies a two-part `base64url(JSON).base64url(signature)` credential using Ed25519 public keys. The required claims include `sub`, `jti`, `kid`, `aud`, `iss`, `iat` and `exp`; the verification path checks the issuer, station audience, key identity, signature and time validity. A process-local `ReplayGuardStore` tracks token identifiers to reject repeated use within the same running instance. This is a custom prototype token format, not the standard three-part JWS Compact Serialization. The production signing/issuance mechanism and distributed replay handling remain outside the delivered module.

5.3 Access Coordination and Local Policy

`AccessCoordinator.handle_scan(ScanInput)` implements the access workflow and returns an `AccessAttempt`. The coordinator verifies the QR, evaluates permissions, persists an allowed attempt *before* issuing an unlock command, and distinguishes controller acknowledgement from confirmed physical passage. Only the confirmed-passage branch creates an `AttendanceRecord` and persists it with an outbox entry. The `AccessPolicyEvaluator` evaluates subject, gate, direction, grant validity and local policy snapshot age. This write ordering reflects the implemented A2 working assumption for OD-04/OD-05; the broader policy questions remain pending formal architectural resolution.

5.4 Turnstile, Persistence, and Offline Synchronization

The turnstile is represented by a deterministic mock with separate unlock and passage outcomes. SQLite repositories implement access-attempt persistence and atomic attendance/outbox writes. `SyncService` processes pending batches while `SyncRetryWorker` schedules retry, and a simulated Hub receiver supports connectivity-loss, lost-ACK and conflicting-ID scenarios. The backoff design uses 5, 10, 20, 40 and 60 second base delays with jitter configured within 0.8–1.0 in the reviewed implementation.

5.5 Source Code Repository and Demonstration Availability

The Edge prototype source, architecture documentation, shared contracts, tests and demonstration scripts are available in the group's GitHub repository:

https://github.com/theDoub/SA-tmc-attendance-access-control

The repository includes `tests/integration/test_end_to_end_access.py` for the combined QR–policy–gate–passage–attendance–outbox–sync path and `scripts/demo_sync.py` for simulated disconnection, reconnection, lost acknowledgement and repeated delivery. The repository contains the executable prototype and its automated tests. Physical hardware and production Hub integration are outside the evaluated scope.

6 Verification and Evaluation

6.1 Testing Strategy

Verification covers QR and policy unit tests, SQLite and synchronization integration tests, and end-to-end scenarios spanning authorization, gate simulation, passage confirmation, local persistence, and event delivery. A dedicated offline integration test exercises the full Edge access path while the simulated Hub is disconnected. These tests use simulated integrations and do not establish production hardware or HR/Payroll behavior.

6.2 Repository Verification Status

Verification level	**Evidence**	**Interpretation / limitation**
Static checks	Ruff lint/format, workflow and secret-check scripts	Passing CI at the referenced revision; static checks do not measure runtime reliability
Unit testing	QR and access-core test suites under `tests/unit/`	Checks selected domain/security behaviors, not physical devices
Persistence and sync integration	SQLite WAL, atomic outbox, retry/ACK and duplicate scenarios under `tests/integration/`	Uses local database and simulated receiver rather than production Hub
Combined integration	`test_end_to_end_access.py`	Exercises the implemented Edge components together in a controlled scenario
Offline access integration	`test_offline_scenarios_pending.py`: full Edge scan while Hub disconnected	Confirms local attendance persistence and a pending outbox event without Hub delivery
Production interfaces	Real turnstile, mobile issuer, Hub and HR/Payroll	Not implemented or verified in A2 prototype

Table 6: Verification evidence and implementation boundaries for the merged A2 source.

Verification artifacts are maintained in the repository under `tests/` and `.github/workflows/ci.yml`.

6.3 Latest Reported Automated Test Results

The most recent local validation reported by the team used `scripts/validate.ps1` after replacing the previously skipped offline placeholder with the integration test `test_network_disconnection_keeps_gate_working`. The test constructs a signed Ed25519 QR credential, evaluates a local access policy, completes a mock turnstile passage, and checks the persisted SQLite attendance and pending outbox records while the simulated Hub is disconnected. An attempted synchronization raises a connection error without removing the local event or delivering it to the receiver.

Validation item	**Reported result**
Pytest suite	291 passed; 0 failed; 0 skipped
Ruff lint and format	Passed; 81 Python files formatted
Workflow and secret checks	Passed; no secret-scan findings
Remaining warning	One deprecation warning associated with Starlette TestClient

Table 7: Local automated validation results reported after the offline integration-test update.

These figures describe the team's reported local test run; they should be associated with the submitted commit and CI run before final submission. The result does not demonstrate compliance with the production response-time, hardware reliability, or cross-kiosk security requirements.

6.4 Three Representative Interactive Demo Scenarios

The interactive web prototype provides a visual interface to the integrated Edge components. Three representative scenarios are selected for report evidence because they cover the main attendance workflow and both M2 issues. Each screenshot should be captured from a completed demo run and show the relevant gate status, recorded data, or synchronization state. Automated test evidence is maintained separately in the repository.

ID	**Scenario**	**Expected result and evidence**
TC-01	Successful attendance	Valid signed QR and authorized policy lead to mock passage confirmation, one attendance record, and an outbox event. Capture gate status and the stored attendance/outbox records.
TC-02	Replay rejection	Reusing the same previously accepted QR token is rejected by the local replay guard. Capture the rejection reason and verify that no second attendance record is created.
TC-03	Offline access and reconnection	With the simulated Hub offline, an authorized passage remains recorded locally and its outbox entry stays PENDING. After reconnecting and receiving an acknowledgement, it becomes SYNCED. Capture the two synchronization states.

Table 8: Three selected interactive demo scenarios and evidence to be captured.

TC-01: Successful attendance. The intended evidence is the confirmed gate passage and the corresponding SQLite attendance/outbox entries.

Demo evidence screenshot to be added

`Images/demo/tc01-successful-attendance.png`

Figure 10: Interactive demo evidence for TC-01: successful attendance and local persistence.

TC-02: Replay rejection. The intended evidence is a second scan of the same token producing a replay rejection without a duplicate attendance entry.

Demo evidence screenshot to be added

`Images/demo/tc02-replay-rejection.png`

Figure 11: Interactive demo evidence for TC-02: replayed QR token rejected.

TC-03: Offline operation and reconnect. The intended evidence is a local PENDING outbox event while offline followed by a SYNCED status after successful simulated Hub acknowledgement. A two-state composite screenshot may be used.

Demo evidence screenshot to be added

`Images/demo/tc03-offline-reconnect.png`

Figure 12: Interactive demo evidence for TC-03: pending offline event and synchronization after reconnect.

The corresponding automated coverage is provided by `tests/integration/test_web_demo.py`, `tests/integration/test_end_to_end_access.py`, and `tests/integration/test_offline_scenarios_pending.py`. The screenshots are illustrative evidence of local demonstration runs, while the automated tests assert the recorded outcomes. The demo uses a mock gate and a simulated Hub; physical-device and production-network behavior are outside its verification scope.

6.5 Requirements Traceability

Requirement / issue	Implementation or design evidence	Verification limitation
Connectivity loss	Offline Edge integration test, SQLite WAL, pending outbox, reconnect and simulated receiver	Mock gate and simulated Hub; no production WAN verification
Static QR exposure	Signed short-lived design; Ed25519 verification and local replay checks	Cross-kiosk replay and issuance unresolved
Attendance integrity	Separate attempt, ACK, passage, attendance and outbox	Real sensor and power-failure behavior not tested
Kiosk latency ≤ 2 s	Local decision path and timeout configuration	No final measured workload benchmark supplied
Audit / privacy	Local access-attempt metadata, payload redaction and scoped ports	Full governance/enterprise controls out of scope
Module implementation	QR, coordinator, policy, persistence, sync and interactive demo tests	Test results require linkage to the submitted code revision

Table 9: M2 requirement-to-design and verification traceability.

7 Architecture Decisions and SOLID Principles

7.1 ADRs Carried Forward from Assignment 1

The A1 report accepted four architectural decision records: ADR-001 (macro/micro architectural styles), ADR-002 (dynamic cryptographically signed QR verification), ADR-003 (offline-first Edge resilience with local cache and transactional outbox), and ADR-004 (legacy HR/Payroll integration strategy) [2]. A2 retains the architectural intent while implementing a restricted Edge module. A1 acceptance of an ADR should not be conflated with a later implementation detail being fully implemented and verified.

7.2 Architectural Ambiguities and Working Assumptions

ID	Question requiring governance	Status for report
OD-01/02	QR signing/issuance boundary and final token claims/format	Pending; Edge verifies a custom two-part prototype token
OD-04	Ordering of durable attempt, unlock and attendance/outbox	Pending; implemented assumption: attempt before unlock, attendance after passage
OD-05	Whether controller ACK can substitute for physical passage	Pending; prototype uses confirmed passage only
OD-08	Replay state durability and coordination across kiosks	Pending; process-local in-memory prototype
OD-10	Production Edge–Hub authentication	Pending; outside the prototype
OD-11	Sequence scope (per kiosk or gate) and uniqueness under concurrent in-flight scans	Pending; the SQLite allocator uses `MAX(source_sequence)+1`

Table 10: A1 ambiguities that must not be silently marked resolved.

7.3 SOLID in the Implemented Edge Module

Principle	Example	Explanation and evidence boundary
SRP	Access coordination, policy, QR and storage	Cryptographic checks, policy decisions, gate orchestration and persistence have distinct primary responsibilities
OCP	QR, gate and sync ports	Alternative adapters can be introduced through stable protocols; actual substitutability depends on contract tests
LSP	Gate ACK/passage contract	Mock/real adapters must preserve the distinction; production vendor substitution has not been verified
ISP	Task-specific interfaces	Clients use interfaces focused on needed operations rather than one omnibus service
DIP	Coordinator and sync service	Application use cases depend on domain interfaces; concrete implementations are composed at the configuration boundary

Table 11: SOLID mapping from design principles to A2 implementation elements.

7.4 SOLID Reflection: Benefits and Challenges

The SOLID principles emphasize responsibility boundaries, substitutability and dependency management [4]. In this implementation, they made independently developed components testable and allowed QR, policy, hardware and persistence collaborators to be replaced with deterministic substitutes. This supported parallel implementation: the access core could be unit-tested against mock ports, while SQLite and synchronization logic could be tested without a physical gate or production Hub. Narrow interfaces reduced the need to import infrastructure concerns into the domain and application layers.

The main practical challenge was not implementing classes in isolation but *agreeing on shared semantics*. During integration, the Access Core and persistence branches used incompatible attendance metadata names (`controller_ack` versus `controller_outcome`) and disagreed about the presence of `source_sequence`. A dedicated integration fix reconciled these contracts and added an end-to-end test. This illustrates that Dependency Inversion makes components replaceable only when the abstract contracts are stable and shared consistently.

Another challenge was the tension between strict safety rules and incomplete external systems: denying on missing or stale policy improves security but can reduce access availability; passage confirmation protects attendance correctness but extends the observed processing window; local replay prevention is simple to simulate but weak across restarts or multiple kiosks. SOLID improves modularity, maintainability and extensibility, yet it does not eliminate the need for architectural trade-off decisions, realistic test scenarios and documentation governance.

8 Conclusion and Known Limitations

Assignment 2 extends the A1 TMC architecture by refining QR security and offline transaction handling and by realizing the Edge Gate Access Processing module as a Python prototype. The implemented classes, persistence contracts, simulations and unit/integration tests provide bounded evidence that the selected architecture can be decomposed and exercised without a full production Hub. The three architecture views explain static decomposition, runtime communication and deployment allocation at different abstraction levels.

The delivered prototype does not implement mobile QR issuance, real turnstile hardware, production Hub ingestion, HR/Payroll reconciliation or globally coordinated replay protection. The current sequence allocator uses `MAX(source_sequence)+1` over stored attendance rows, which can return the same value to concurrent in-flight scans before either record is inserted; uniqueness and ordering across such scans are not established. The prototype tests do not establish production-level correctness or compliance with all non-functional requirements. Performance and availability targets require representative measurements; the remaining architectural decisions are open.

References

[1] HCMUT, *Software Architecture (CO3017): Assignment – M2*, course assignment specification, 2026.

[2] Group 6, *Software Architecture (CO3017): Assignment 1 – Requirement Analysis & Software Design*, HCMUT, October 2026.

[3] P. Clements et al., *Documenting Software Architectures: Views and Beyond*, 2nd ed., Addison-Wesley, 2011.

[4] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Pearson, 2017.

[5] M. Richards and N. Ford, *Fundamentals of Software Architecture*, O'Reilly Media, 2020.

[6] Group 6, *TMC Attendance and Access Control – A2 Source Repository*, main branch, (14 November 2026). https://github.com/theDoub/SA-tmc-attendance-access-control. Accessed 14 November 2026.

A AI Tool Usage Declaration

A.1 Nguyen Tran Tuan Khai — Architecture Views and Interface Contracts

1. **AI tools used:** [TO CONFIRM BY MEMBER]
2. **Purpose of AI assistance:** [TO CONFIRM BY MEMBER]
3. **Representative prompts/requests:** [TO CONFIRM BY MEMBER]
4. **AI-assisted content/outputs and human verification:** [TO CONFIRM BY MEMBER]

A.2 Nguyen Ngoc Kim — Edge Access Core and Policy Evaluation

1. **AI tools used:** [TO CONFIRM BY MEMBER]
2. **Purpose of AI assistance:** [TO CONFIRM BY MEMBER]
3. **Representative prompts/requests:** [TO CONFIRM BY MEMBER]
4. **AI-assisted content/outputs and human verification:** [TO CONFIRM BY MEMBER]

A.3 Nguyen Nhat Hieu — Issue Analysis and QR Verification

1. **AI tools used:** [TO CONFIRM BY MEMBER]
2. **Purpose of AI assistance:** [TO CONFIRM BY MEMBER]
3. **Representative prompts/requests:** [TO CONFIRM BY MEMBER]
4. **AI-assisted content/outputs and human verification:** [TO CONFIRM BY MEMBER]

A.4 Bui Doan Minh Nhat — Persistence, Turnstile and Synchronization

1. **AI tools used:** [TO CONFIRM BY MEMBER]
2. **Purpose of AI assistance:** [TO CONFIRM BY MEMBER]
3. **Representative prompts/requests:** [TO CONFIRM BY MEMBER]

4. AI-assisted content/outputs and human verification: [TO CONFIRM BY MEMBER]

A.5 Vu Quoc Viet — ADR Refinement, Integration and Report Assembly

1. AI tools used: [TO CONFIRM BY MEMBER]

2. Purpose of AI assistance: [TO CONFIRM BY MEMBER]

3. Representative prompts/requests: [TO CONFIRM BY MEMBER]

4. AI-assisted content/outputs and human verification: [TO CONFIRM BY MEMBER]